IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Göksel Keser
2025

Outline

EXECUTIVE SUMMARY

INTRODUCTION

METHODOLOGY

RESULTS

CONCLUSION

APPENDIX

Executive Summary

This project involves SpaceX utilizing data from the Falcon 9 rocket to assess the efficiency of its first stage landings. To determine the outcome, the following methodologies are used:

- **Data Collection:** Aggregating relevant data from credible sources.
- **Data Wrangling:** Cleaning and standardizing the dataset for accuracy.
- **Exploratory Data Analysis:** Utilizing statistical tools to identify patterns and correlations.
- **Data Visualization:** Employing visual aids to effectively highlight insights.
- **Machine Learning Prediction:** Building predictive models to forecast landing success and optimize launch strategies.

Launch outcomes and proximities are visualized with Folium maps, while charts highlight success ratios, including payload mass range achievements. Machine learning models like logistic regression, SVM, Decision Tree, and k-nearest neighbors are used for predictive analysis, pinpointing the top-performing model.

Introduction

In the rapidly evolving landscape of commercial space exploration, cost-effective rocket launches are a pivotal factor driving competitive edge. SpaceX, with its Falcon 9 rocket, stands as a significant player, offering launches at $62 million—a stark contrast to other providers charging upwards of $165 million, largely due to its reusable first stage technology. This project aims to harness Falcon 9 data to evaluate landing efficiencies, enabling to strategize effectively and potentially bid alongside SpaceX for rocket launch contracts.

The project seeks to identify key factors that influence the success of the Falcon 9's first stage landing. It aims to understand how various rocket parameters impact the likelihood of a successful landing. Ultimately, the goal is to develop predictive models that can enhance strategic planning and ensure cost-effective launch operations.

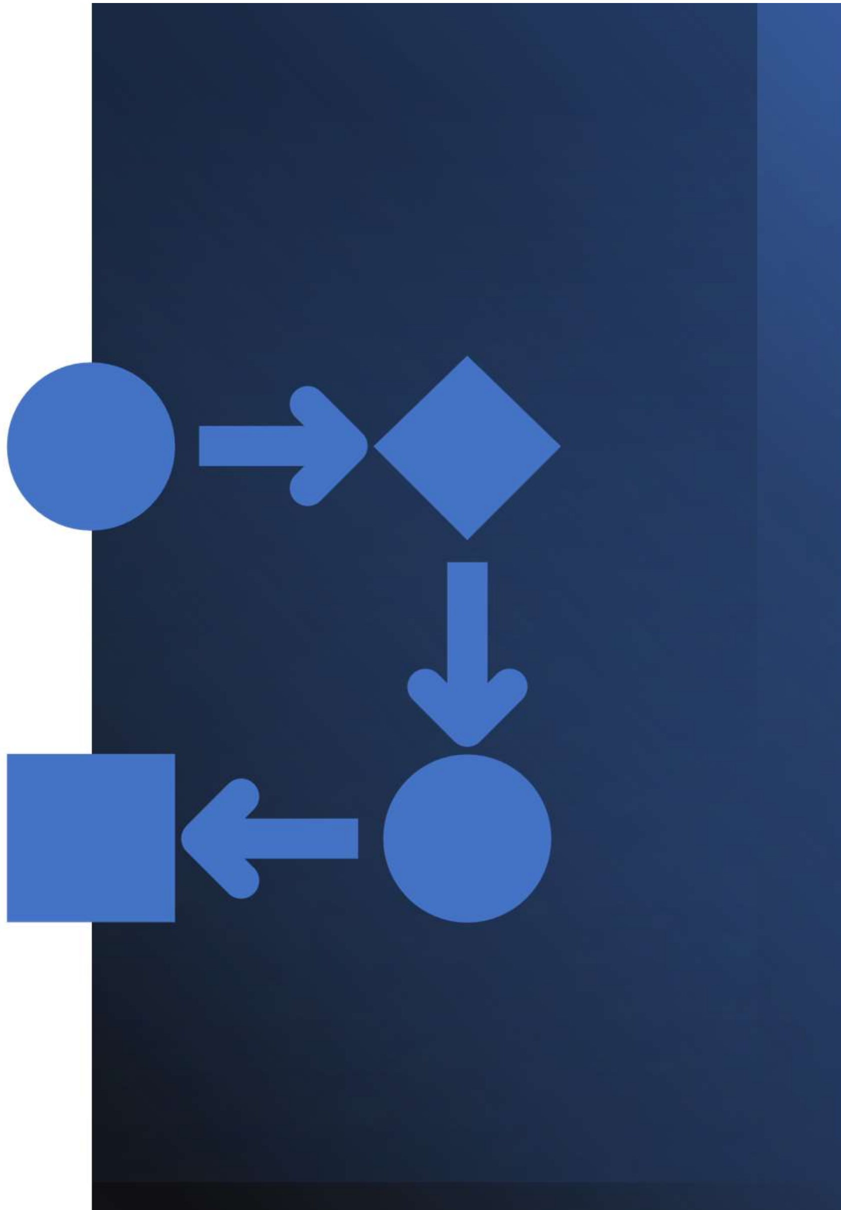
Section 1

Methodology

Methodology

This project followed a structured methodology encompassing several key steps:

- **Data Collection:** Utilized the SpaceX REST API and web scraping techniques from Wikipedia to gather comprehensive data sets.
- **Data Wrangling:** Prepared the data for analysis by cleaning and encoding it for machine learning applications, while removing irrelevant columns to optimize performance.
- **Exploratory Data Analysis (EDA):** Conducted EDA using SQL and visualization techniques to uncover patterns and insights within the data.
- **Visual Analytics:** Leveraged interactive tools such as Folium and Plotly Dash to perform advanced visual analytics.
- **Predictive Analysis:** Utilized statistical techniques, including logistic regression and support vector machines, to develop predictive models.



Data Collection – Scraping - Flow

Request Data → Process Response & Extract Launch Information → Structure Data → Filter Specific Launches → Handle Missing Values → Data Export

Data Collection – SpaceX API

1. **Request Data:**

```
spacex_url="https://api.spacexdata.com/v4/launches/past"
```

```
response = requests.get(spacex_url)
```

2. **Process Response & Extract Launch Information:**

```
# Use json_normalize meethod to convert the json result into a dataframe
json_data = response.json()
df_launch_data = pd.json_normalize(json_data)
```

3. **Structure Data:**

```
# Call getBoosterVersion
getBoosterVersion(data)
```

```
# Call getLaunchSite
getLaunchSite(data)
```

```
# Call getPayloadData
getPayloadData(data)
```

```
# Call getCoreData
getCoreData(data)
```

```
launch_dict = {'FlightNumber': list(data['flight_number']),
'Date': list(data['date']),
```

4. **Filter Specific Launches:**

```
# Hint data['BoosterVersion']!='Falcon 1'
data_falcon9 = launch_df[launch_df['BoosterVersion']  == 'Falcon 9']

print(data_falcon9.head())
```

5. **Handle Missing Values:**

```
# Calculate the mean value of PayloadMass column
mean_payload_mass = data_falcon9['PayloadMass'].mean()

# Replace the np.nan values with its mean value
data_falcon9['PayloadMass'] = data_falcon9['PayloadMass'].fillna(mean_payload_ma

print(data_falcon9['PayloadMass'].isnull().sum())
```

6. **Data Export:**

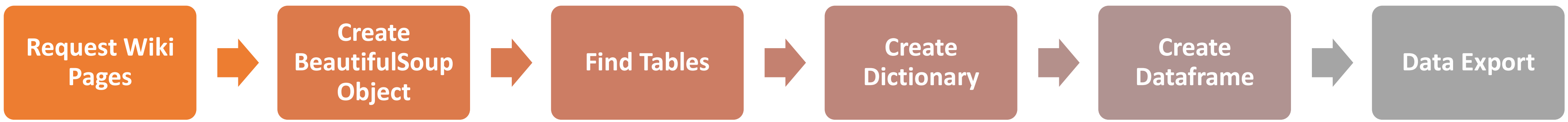
```
data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

Data Collection – SpaceX API - Output

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude
4	1	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	None None	1	False	False	False	None	1.0	0	B0003	-80.577366	28.561857
5	2	2012-05-22	Falcon 9	525.0	LEO	CCSFS SLC 40	None None	1	False	False	False	None	1.0	0	B0005	-80.577366	28.561857
6	3	2013-03-01	Falcon 9	677.0	ISS	CCSFS SLC 40	None None	1	False	False	False	None	1.0	0	B0007	-80.577366	28.561857
7	4	2013-09-29	Falcon 9	500.0	PO	VAFB SLC 4E	False Ocean	1	False	False	False	None	1.0	0	B1003	-120.610829	34.632093
8	5	2013-12-03	Falcon 9	3170.0	GTO	CCSFS SLC 40	None None	1	False	False	False	None	1.0	0	B1004	-80.577366	28.561857

URL : https://github.com/GokKes/IBM---SpaceX-Capstone-Project/blob/main/1%20-%20SpaceX%20Data%20Collection.ipynb

Data Collection – Scraping - Flow



Data Collection - Scraping

1. Request Wiki Pages

```
static_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Fa
```

```
# use requests.get() method with the provided static_url
# assign the response to a object
response = requests.get(static_url)
```

2. Create BeautifulSoup Object

```
# Use BeautifulSoup() to create a BeautifulSoup object from a response text cont
soup = BeautifulSoup(response.text, 'html.parser')
```

3. Find Tables

```
# Use the find_all function in the BeautifulSoup object, with element type `tabl
# Assign the result to a list called `html_tables`
html_tables = soup.find_all('table')
print(f"Number of tables found: {len(html_tables)}")
```

```
# Let's print the third table and check its content
first_launch_table = html_tables[2]
print(first_launch_table)
```

4. Create Dictionary

```
launch_dict= dict.fromkeys(column_names)

# Remove an irrelvant column
del launch_dict['Date and time ( )']

# Let's initial the launch_dict with each value to be an empty list
launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
```

5. Create Dataframe

```
df=pd.DataFrame(launch_dict)
df.head()
```

6. Export Data

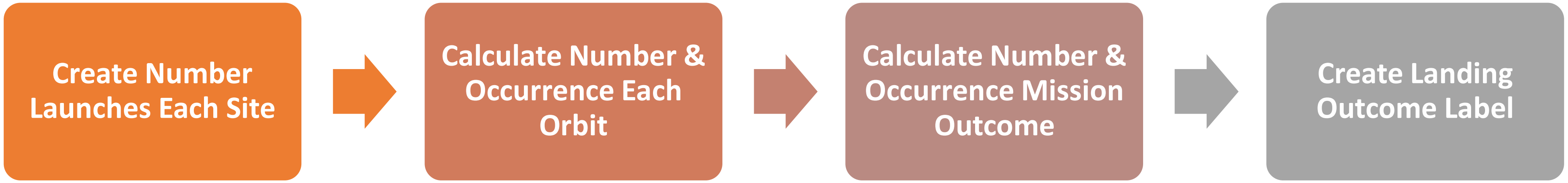
```
df.to_csv('spacex_web_scraped.csv', index=False)
```

Data Collection – Scraping - Output

	Flight No.	Launch site	Payload	Payload mass	Orbit	Customer	Launch outcome	Version Booster	Booster landing	Date	Time
0	1	CCAFS	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success\n	F9 v1.07B0003.18	Failure	4 June 2010	18:45
1	2	CCAFS	Dragon	0	LEO	NASA	Success	F9 v1.07B0004.18	Failure	8 December 2010	15:43
2	3	CCAFS	Dragon	525 kg	LEO	NASA	Success	F9 v1.07B0005.18	No attempt\n	22 May 2012	07:44
3	4	CCAFS	SpaceX CRS-1	4,700 kg	LEO	NASA	Success\n	F9 v1.07B0006.18	No attempt	8 October 2012	00:35
4	5	CCAFS	SpaceX CRS-2	4,877 kg	LEO	NASA	Success\n	F9 v1.07B0007.18	No attempt\n	1 March 2013	15:10

URL : https://github.com/GokKes/IBM---SpaceX-Capstone-Project/blob/main/2%20-%20SpaceX%20Web%20Scraping.ipynb

Data Wrangling - Flow



Data Wrangling

1. Create Number Launches Each Site

```
# Apply value_counts() on column LaunchSite

df['LaunchSite'].value_counts()
```

2. Calculate Number & Occurrence Each Orbit

```
# Apply value_counts on Orbit column

df['Orbit'].value_counts()
```

3. Calculate Number & Occurrence Mission Outcome

```
# landing_outcomes = values on Outcome column

landing_outcomes = df['Outcome'].value_counts()
landing_outcomes
```

```
for i,outcome in enumerate(landing_outcomes.keys()):
    print(i,outcome)
```

```
bad_outcomes=set(landing_outcomes.keys()[[1,3,5,6,7]])
bad_outcomes
```

4. Create Landing Outcome Label

```
# landing_class = 0 if bad_outcome
# landing_class = 1 otherwise
landing_class = []

for i in df['Outcome']:
    if i in set(bad_outcomes):
        landing_class.append(0)
    else:
        landing_class.append(1)
```

```
df['Class']=landing_class
df[['Class']].head(8)
```

Data Wrangling - Output

```
df.head(5)
```

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude	Class
0	1	2010-06-04	Falcon 9	6104.959412	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.577366	28.561857	0
1	2	2012-05-22	Falcon 9	525.000000	LEO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.577366	28.561857	0
2	3	2013-03-01	Falcon 9	677.000000	ISS	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.577366	28.561857	0
3	4	2013-09-29	Falcon 9	500.000000	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.610829	34.632093	0
4	5	2013-12-03	Falcon 9	3170.000000	GTO	CCAFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.577366	28.561857	0

URL : https://github.com/GokKes/IBM---SpaceX-Capstone-Project/blob/main/3%20-%20%20SpaceX%20Data%20Wrangling.ipynb

EDA with Data Visualization

These steps were undertaken to craft various visualizations:

- Visualization relationship between Flight Number and Launch Site with Scatter Plot
- Visualization relationship between Payload Mass and Launch Site with Scatter Plot
- Visualization relationship between success rate of each orbit type with Bar Chart
- Visualization relationship between FlightNumber and Orbit type with Scatter Plot
- Visualization relationship between Payload Mass and Orbit type with Scatter Plot
- Visualization of launch success shown in yearly trend with Line Chart
- Creation of dummy variables to categorical columns with “One Hot Encoding”
- Casting all numeric columns to float64

Scatter plots, bar charts, and line charts are useful tools for visualizing data outcomes. Scatter plots are particularly useful for identifying correlations and outliers among numerical variables. Bar charts excel at comparing discrete categories by clearly representing differences in magnitude and frequency. Line charts, on the other hand, are ideal for illustrating trends over time, effectively highlighting changes and patterns within time-series data.

URL:
https://github.com/GokKes/IBM---SpaceX-Capstone-Project/blob/main/5%20-%20SpaceX%20EDA%20Visualization.ipynb

EDA with SQL

Following SQL queries were performed:

- Display the names of the unique launch sites in the space mission
- Display 5 records where launch sites begin with the string 'CCA'
- Display the total payload mass carried by boosters launched by NASA (CRS)
- Display average payload mass carried by booster version F9 v1.1
- List the date when the first successful landing outcome in ground pad was achieved.
- List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000
- List the total number of successful and failure mission outcomes
- List all the booster_versions that have carried the maximum payload mass. Use a subquery.
- List the records which will display the month names, failure landing_outcomes in drone ship, booster versions, launch_site for the months in year 2015.
- Rank the count of landing outcomes such as Failure or Success between the date 2010-06-04 and 2017-03-20, in descending order.

URL:
https://github.com/GokKes/IBM---SpaceX-Capstone-Project/blob/main/4%20-%20SpaceX%20EDA%20SQL.ipynb

Build an Interactive Map with Folium

To build interactive Map with Folium 3 major tasks were performed:

All launch sites on a map were marked

- Latitude and longitude coordinates were utilized to display each launch site, with circle markers accompanied by pop-up labels identifying each location.

All success and failed launches for each site were marked on a map

- Color markers were utilized to distinguish launch outcomes, with green indicating successful launches and red denoting failures, incorporating the success rate of the launches.

Distances between a launch site to its proximities were calculated

- Distances from each launch site to the nearest coastline, railway, highway, and city were determined to evaluate proximity.

URL:
https://github.com/GokKes/IBM---SpaceX-Capstone-Project/blob/main/6%20-%20SpaceX%20Visual%20Analytics%20Folium.ipynb

Build a Dashboard with Plotly Dash

A Dash application was developed featuring a dropdown menu paired with corresponding charts for enhanced visualization flexibility:

- A dropdown menu encompassing each launch site was created.
- A pie chart illustrating the count of successful launches was developed.
- A slider for selecting the payload range was implemented.
- A scatter chart displaying the correlation between payload and launch success was constructed.
- A callback function linking the site-dropdown as input to the success-pie-chart as output was established.
- A callback function associating the site-dropdown and payload-slider as inputs to the success-payload-scatter-chart as output was created.

URL:
https://github.com/GokKes/IBM---SpaceX-Capstone-Project/blob/main/7%20-%20SpaceX%20Dash%20App.py

Predictive Analysis (Classification)

A predictive analysis was conducted through the following steps:

- Construct a NumPy Array.
- Standardize the data.
- Divide the data into test and training sets.
- Develop a logistic regression model.
- Assess the accuracy using the test data.
- Instantiate a support vector machine object.
- Evaluate the accuracy on the test data.
- Formulate a decision tree classifier.
- Determine the accuracy of the decision tree classifier.
- Create a k-nearest neighbors model.
- Measure the accuracy of the k-nearest neighbors.
- Identify the best-performing method.

URL:
https://github.com/GokKes/IBM---SpaceX-Capstone-Project/blob/main/8%20-%20SpaceX%20Machine%20Learning%20Prediction.ipynb

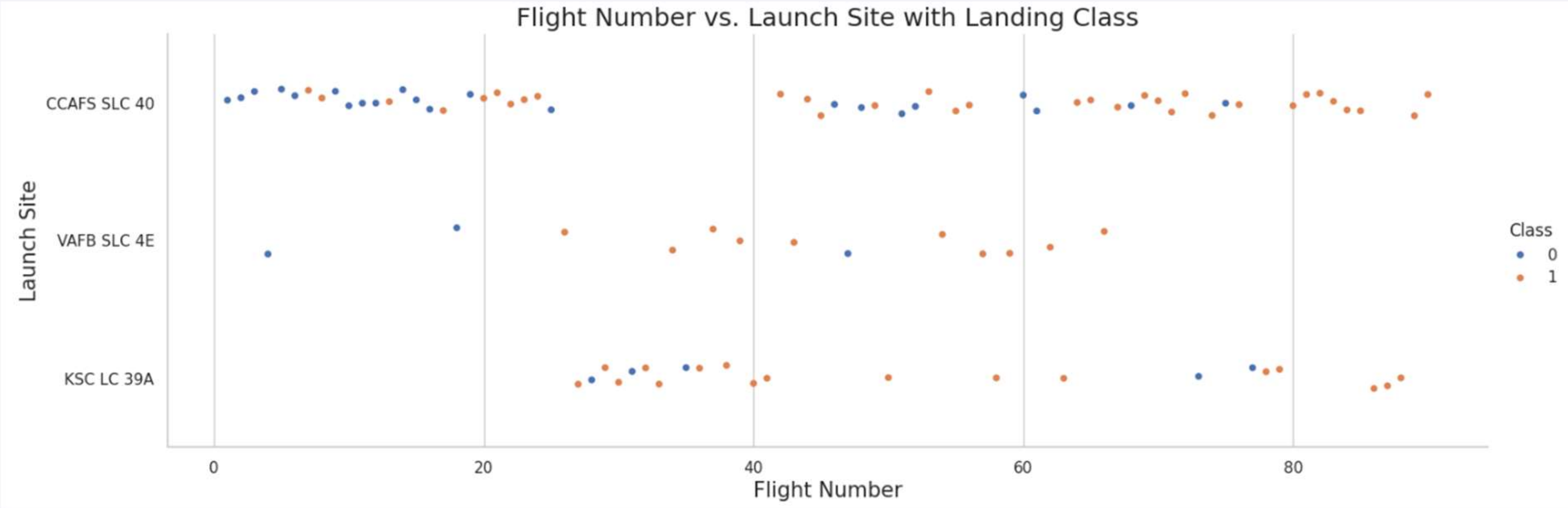
Results

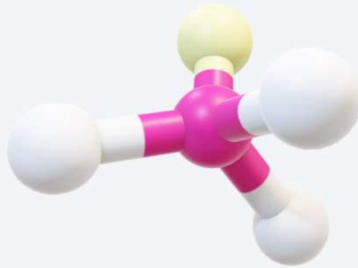
The project showcases exploratory data analysis results, complemented by an interactive analytics demo presented through screenshots, and further enhanced with predictive analysis outcomes

Section 2

Insights drawn from EDA

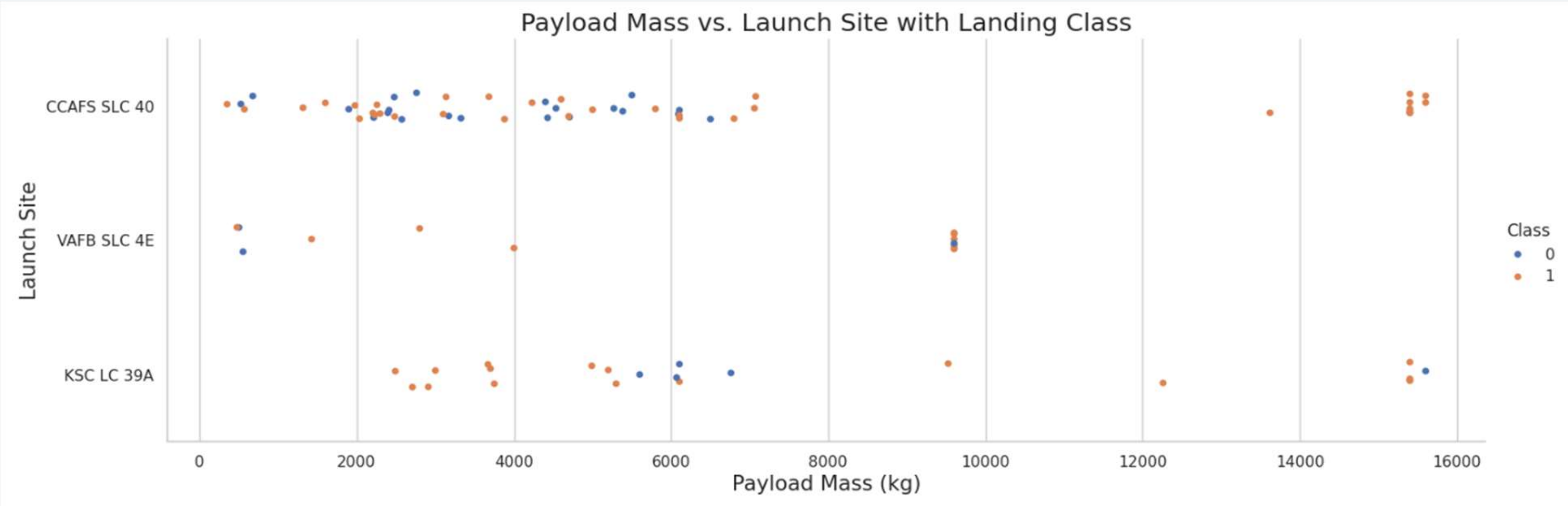
Flight Number vs. Launch Site





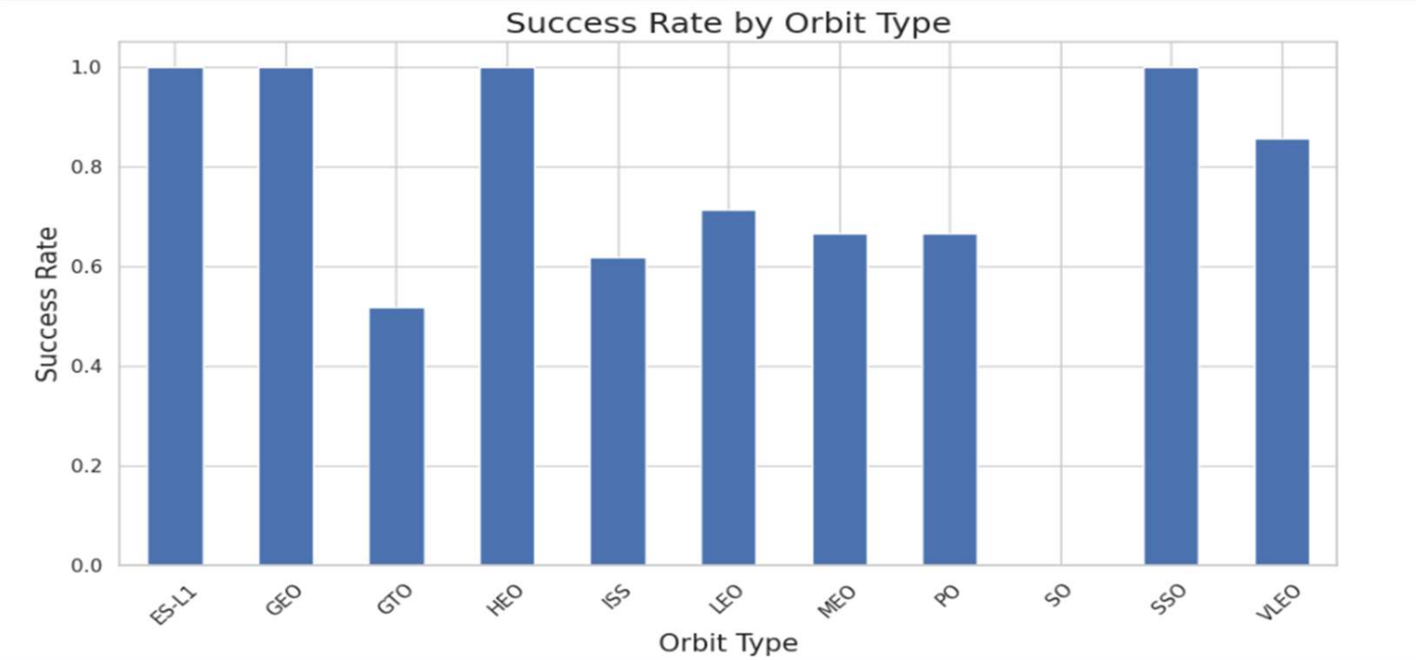
The scatter plot reveals a trend where higher flight numbers correspond with improved success rates, illustrated by blue markers for failed launches and orange markers for successful ones, indicating that newer launches tend to achieve greater success.

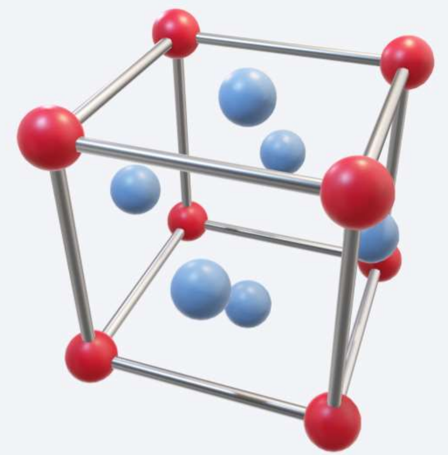
Payload vs. Launch Site



The scatter plot shows a trend where increased Payload Mass is linked to higher success rates, represented by blue markers for unsuccessful launches and orange markers for successful ones.

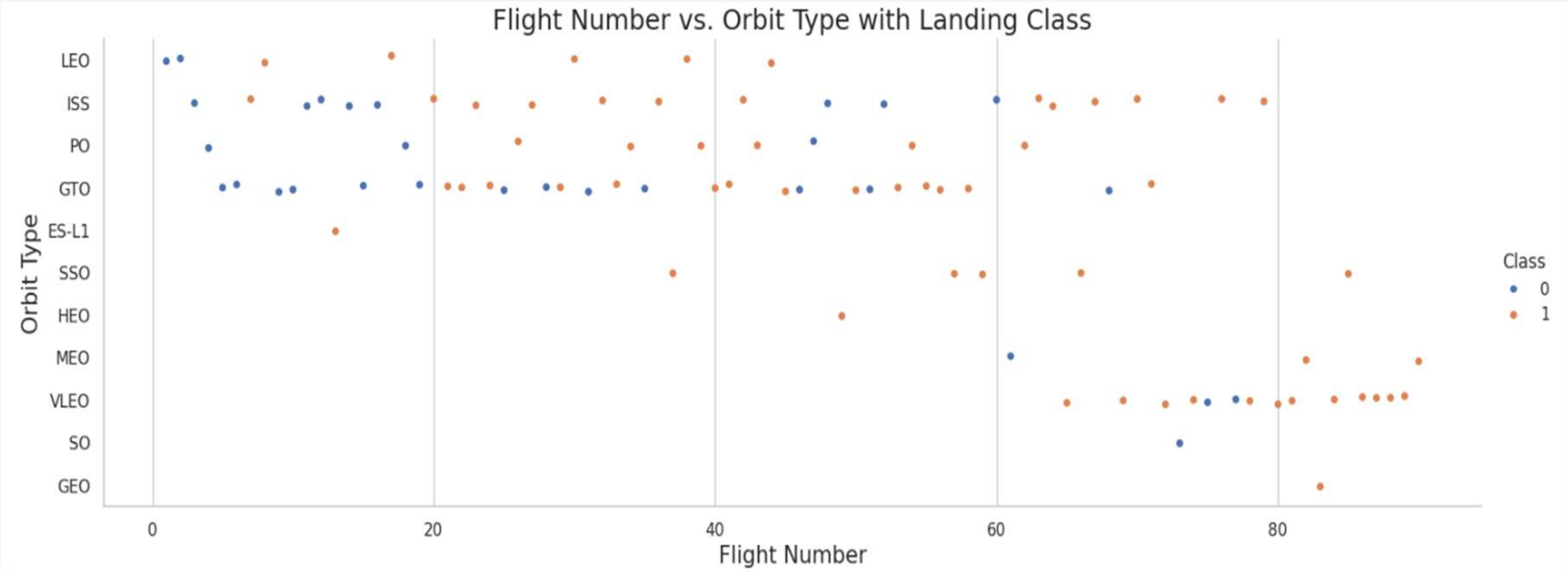
Success Rate vs. Orbit Type

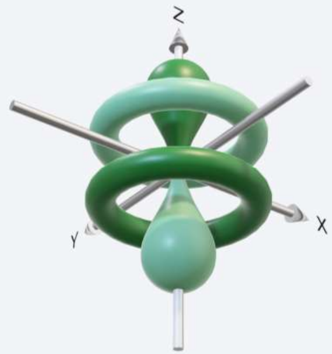




The bar chart depicting success rates by orbit type highlights that ES-L1, GEO, HEO, and SSO orbits boast a perfect success rate of 100%.

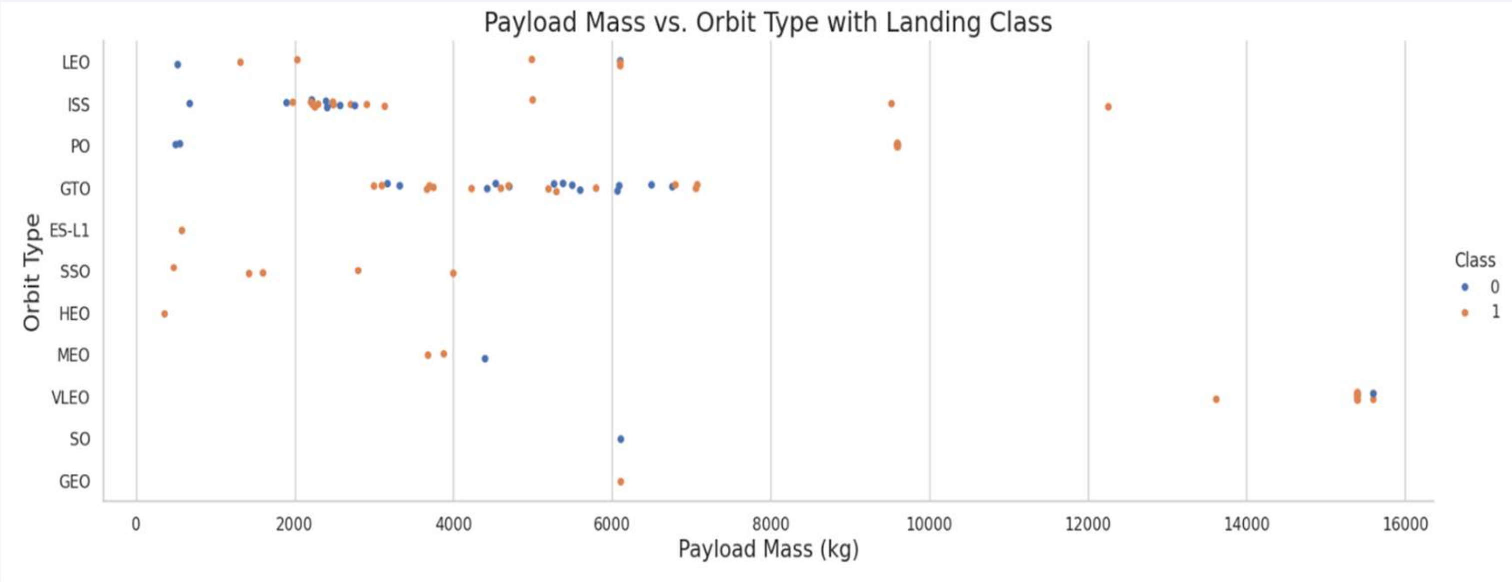
Flight Number vs. Orbit Type

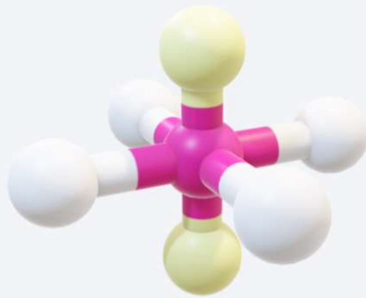




The scatter plot examining Flight Number versus Orbit Type indicates a positive correlation in LEO orbit, where higher flight numbers coincide with increased success, while no discernible trend is observed for GTO orbit.

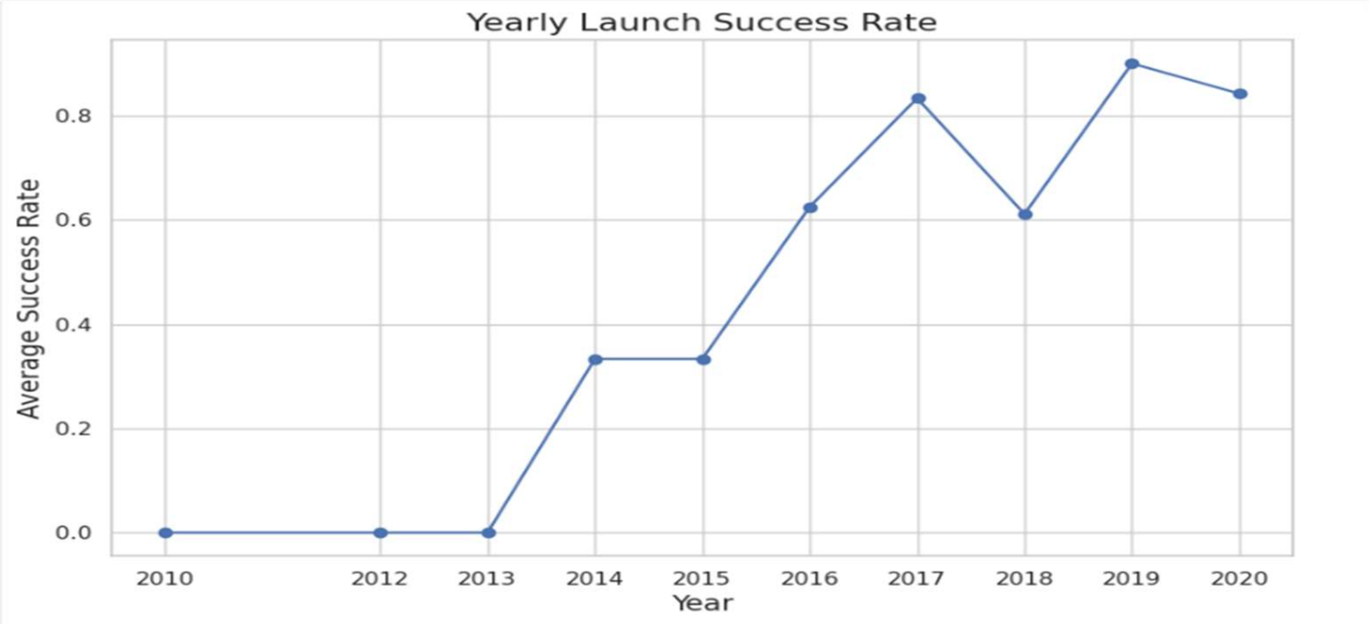
Payload vs. Orbit Type

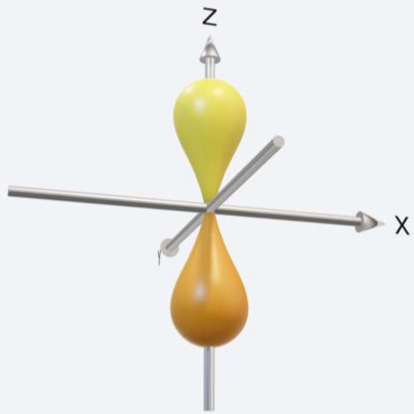




The scatter plot highlights a pattern where increased Payload Mass in LEO, ISS, and PO orbits is associated with higher success rates, whereas no distinct trend exists for GTO orbit.

Launch Success Yearly Trend





The plot illustrates that the success rate steadily rose from 2013 to 2019, experienced a decline between 2019 and 2020, but overall, the trend from 2013 to 2020 remains upward.

All Launch Site Names

The DISTINCT function was utilized to retrieve unique values from the "Launch_Sites" column within the SPACETABLE.

```
%%sql
SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE;
```

```
 * sqlite:///my_data1.db
Done.
```

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Launch Site Names Begin with 'CCA'

Five records were displayed where the launch site names begin with 'CCA', employing the WHERE clause, LIKE pattern matching, and LIMIT function.

```
%%sql
SELECT * FROM SPACEXTABLE
WHERE "Launch_Site" LIKE 'CCA%'
LIMIT 5;
```

```
 * sqlite:///my_data1.db
Done.
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

The SUM function calculated that boosters launched by NASA (CRS) carried a total Payload Mass of 48,213 KG.

```
%%sql
SELECT SUM("PAYLOAD_MASS__KG_") AS Total_Payload_Mass
FROM SPACEXTABLE
WHERE Customer LIKE '%NASA (CRS)%';
```

```
 * sqlite:///my_data1.db
Done.
```

Total_Payload_Mass
48213

Average Payload Mass by F9 v1.1

Employing the AVG() function, the average Payload Mass for the booster version like F9 v1.1x was computed to be approximately 2,535 KG, after rounding up.

```
%%sql
SELECT AVG("PAYLOAD_MASS__KG_") AS Average_Payload_Mass
FROM SPACEXTABLE
WHERE "Booster_Version" LIKE 'F9 v1.1%';
```

```
 * sqlite:///my_data1.db
Done.
```

Average_Payload_Mass
2534.6666666666665

First Successful Ground Landing Date

Using the MIN() function, it was determined that the first successful landing on a ground pad occurred on 22nd December 2015.

```
%%sql
SELECT MIN(Date) AS First_Successful_Pad_Landing
FROM SPACEXTABLE
WHERE "Landing_Outcome" LIKE '%Success (ground pad)%';
```

```
 * sqlite:///my_data1.db
Done.
```

First_Successful_Pad_Landing
2015-12-22

Successful Drone Ship Landing with Payload between 4000 and 6000

A selection was made listing boosters that successfully landed on a drone ship and possessed payload masses ranging between 4,000 and 6,000, accomplished using the WHERE clause, LIKE pattern matching, AND logical operators, along with the < and > operators.

```
%%sql
SELECT "Booster_Version"
FROM SPACEXTABLE
WHERE "Landing_Outcome" LIKE '%Success (drone ship)%'
  AND "PAYLOAD_MASS__KG_" > 4000
  AND "PAYLOAD_MASS__KG_" < 6000;
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Total Number of Successful and Failure Mission Outcomes

The SELECT, COUNT, FROM, GROUP BY functions was utilized to compute the total number of both successful and failed mission outcomes.

```
%%sql
SELECT "Mission_Outcome", COUNT(*) AS Count
FROM SPACEXTABLE
GROUP BY "Mission_Outcome";
```

```
 * sqlite:///my_data1.db
Done.
```

Mission_Outcome	Count
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

A list was created to display the names of boosters that transported the maximum payload mass, utilizing the DISTINCT, FROM, WERE function for unique identification.

```
%%sql
SELECT DISTINCT "Booster_Version"
FROM SPACEXTABLE
WHERE "PAYLOAD_MASS__KG_" = 15600;
```

```
 * sqlite:///my_data1.db
Done.
```

Booster_Version
F9 B5 B1048.4
F9 B5 B1049.4
F9 B5 B1051.3
F9 B5 B1056.4
F9 B5 B1048.5
F9 B5 B1051.4
F9 B5 B1049.5
F9 B5 B1060.2
F9 B5 B1058.3
F9 B5 B1051.6
F9 B5 B1060.3
F9 B5 B1049.7

2015 Launch Records

A list was generated displaying failed landing outcomes on drone ships for the year 2015, including details on booster versions and launch site names, achieved through the use of SELECT, FROM, WHERE, AND, and LIKE functions.

```
%%sql
SELECT
    substr(Date, 6, 2) AS Month,
    "Booster_Version",
    "Launch_Site",
    "Landing_Outcome"
FROM SPACEXTABLE
WHERE substr(Date, 1, 4) = '2015'
  AND "Landing_Outcome" LIKE '%Failure (drone ship)%';
```

```
 * sqlite:///my_data1.db
Done.
```

Month	Booster_Version	Launch_Site	Landing_Outcome
01	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
04	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

A ranking was established based on a count of landing outcomes within the period from 4th June 2010 to 20th March 2017. This was achieved using the functions WHERE, BETWEEN, GROUP BY, and ORDER BY Count DESC to arrange results in descending order.

```
%%sql
SELECT "Landing_Outcome", COUNT(*) AS Count
FROM SPACEXTABLE
WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
GROUP BY "Landing_Outcome"
ORDER BY Count DESC;
```

```
 * sqlite:///my_data1.db
Done.
```

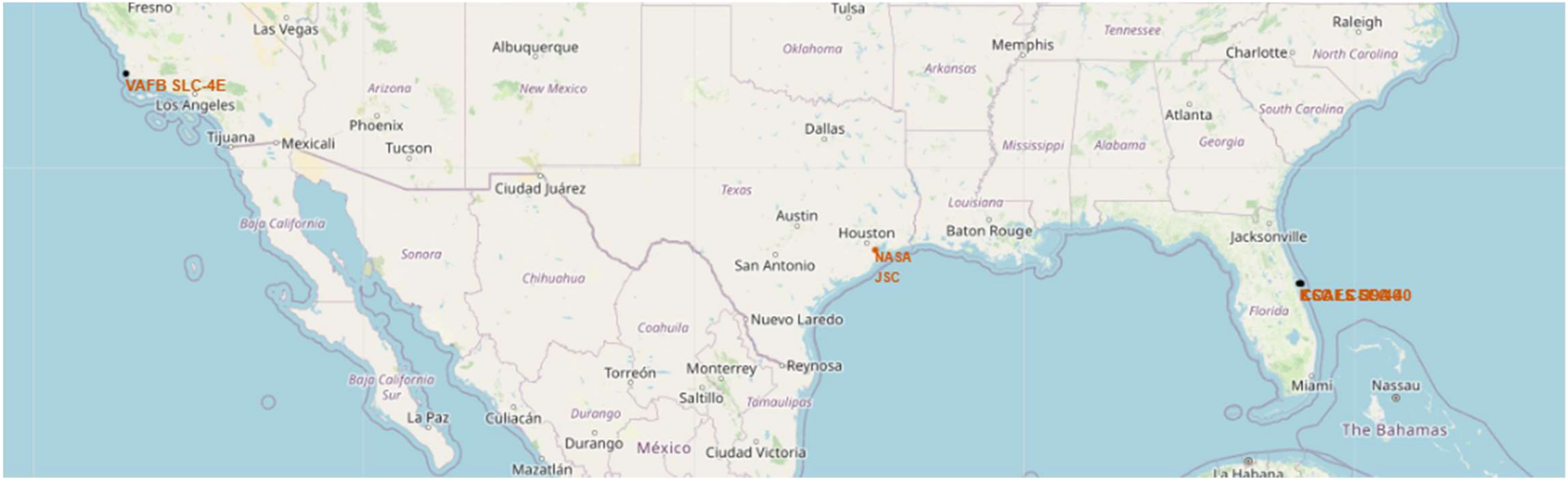
Landing_Outcome	Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Section 3

Launch Sites Proximities Analysis

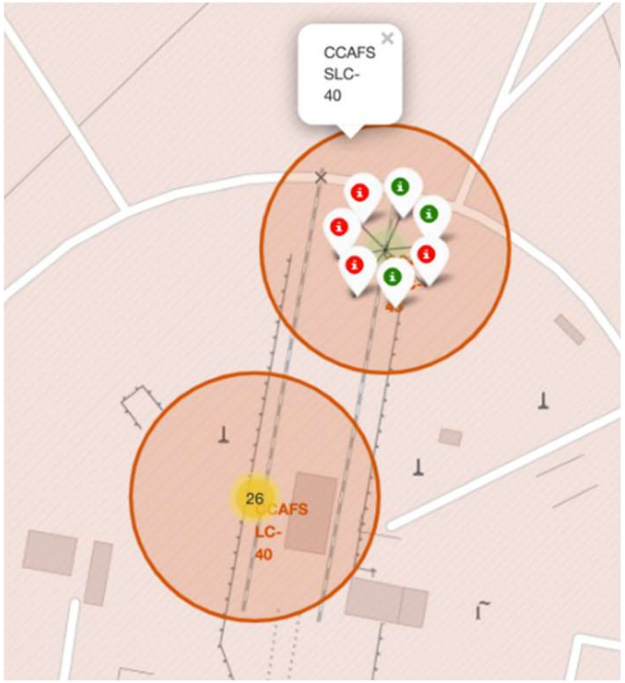
Interactive Folium Map - All Launch Sites

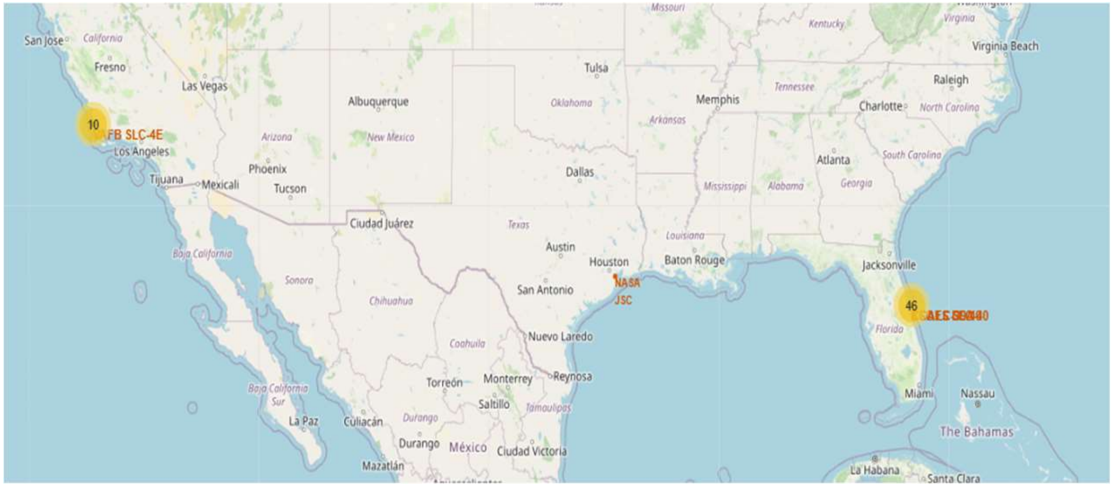
Most launch sites in this project are strategically located near coastlines to enable rockets to launch over oceans, thereby reducing the risk of affecting populated areas with debris.



Color-Labeled Launch Outcomes with Folium Map

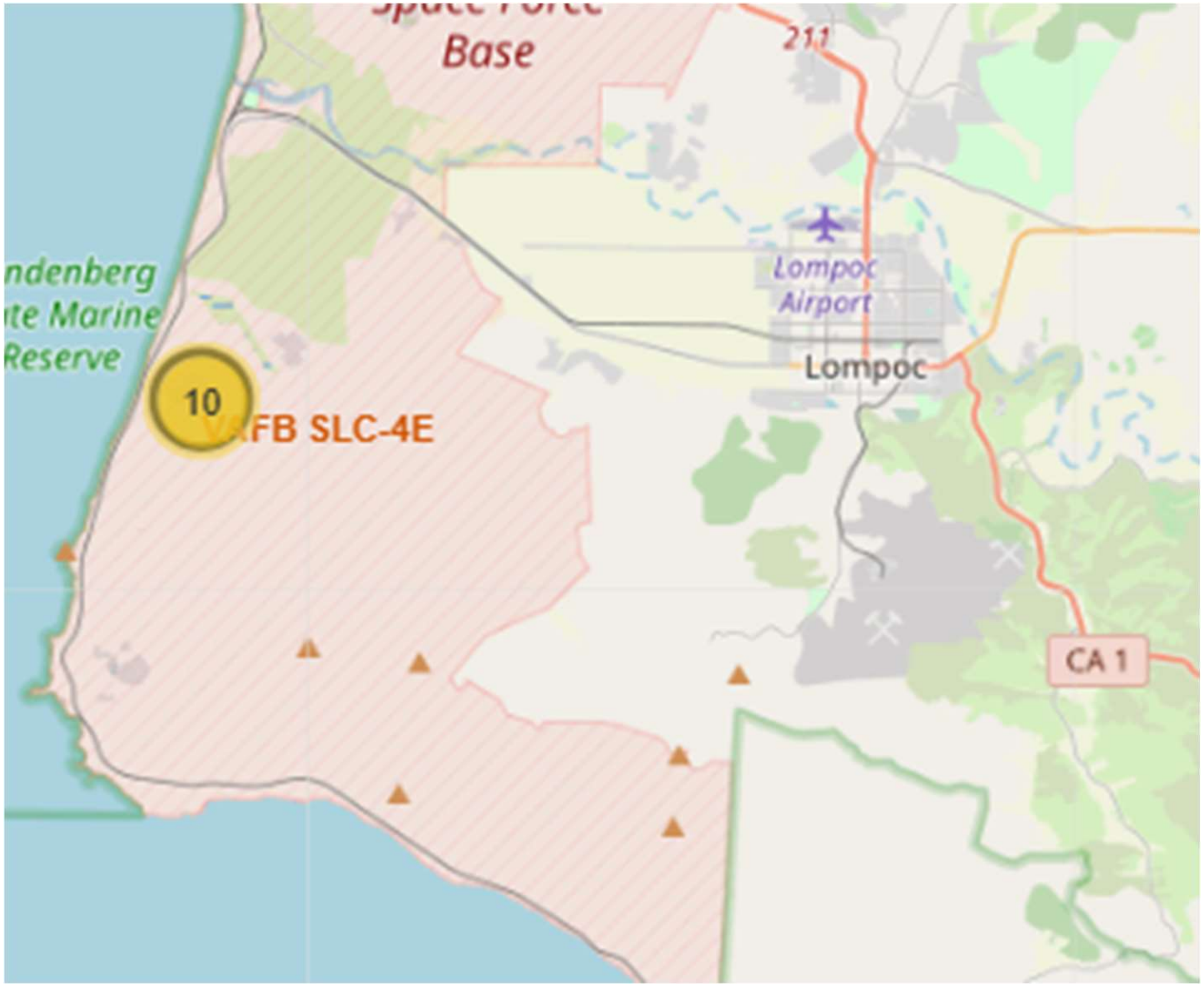
Maps illustrate launch outcomes using color labels, with green indicating successful launches and red representing failed launches.





Launch Site And Its Proximities

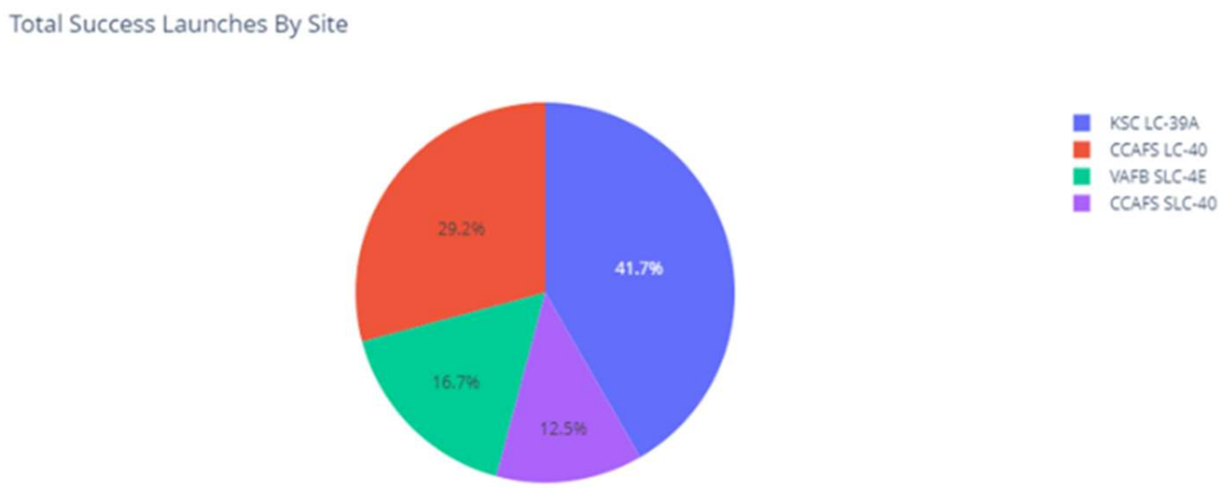
The map displays launch sites along with their proximities to railways, highways, and coastlines.



Section 4

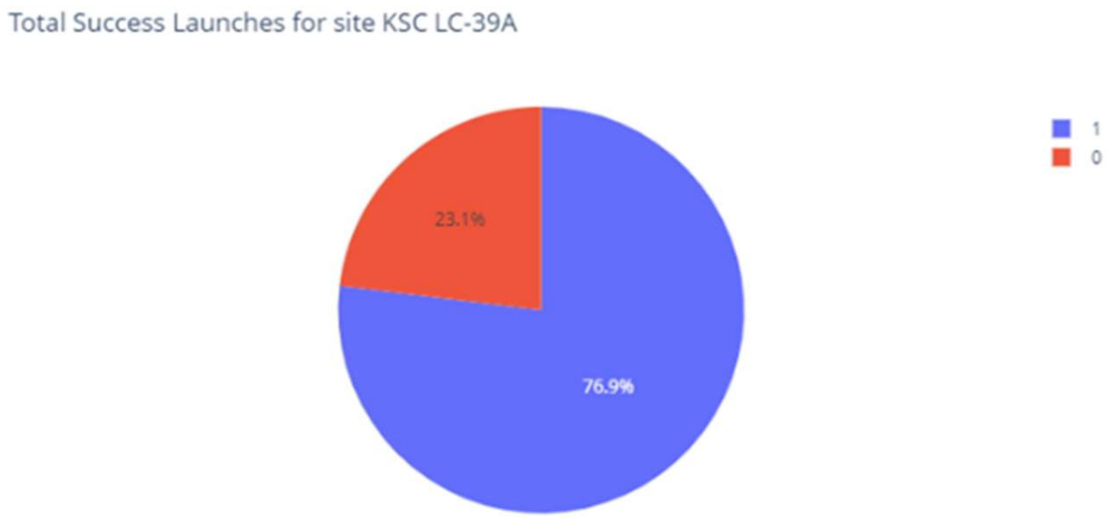
Build a Dashboard with Plotly Dash

Success Launches by Site



KSC LC-39A boasts the highest success rate, while CCAFS SLC-40 has the lowest among the evaluated sites.

Site with highest success ratio



KSC LC-39A distinguishes itself with a leading success ratio of 76.9%.

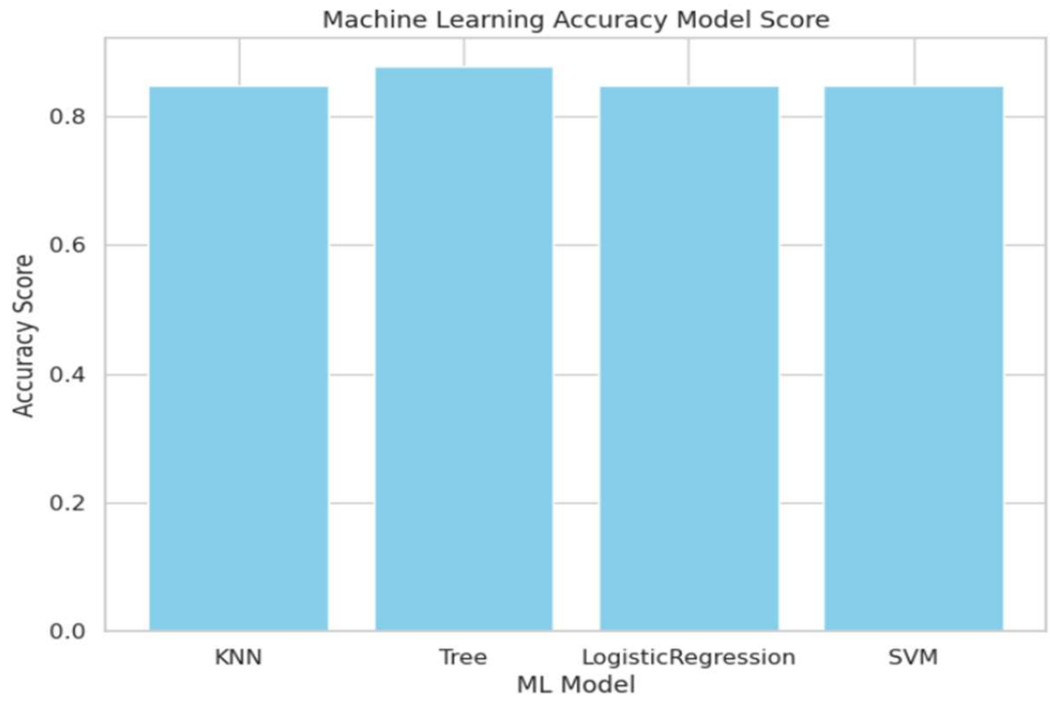
Payload Mass Range Success

The payload mass range of 2,000 to 6,000 KG exhibits the highest success rate, characterized by the most numerous plots within this category.

Section 5

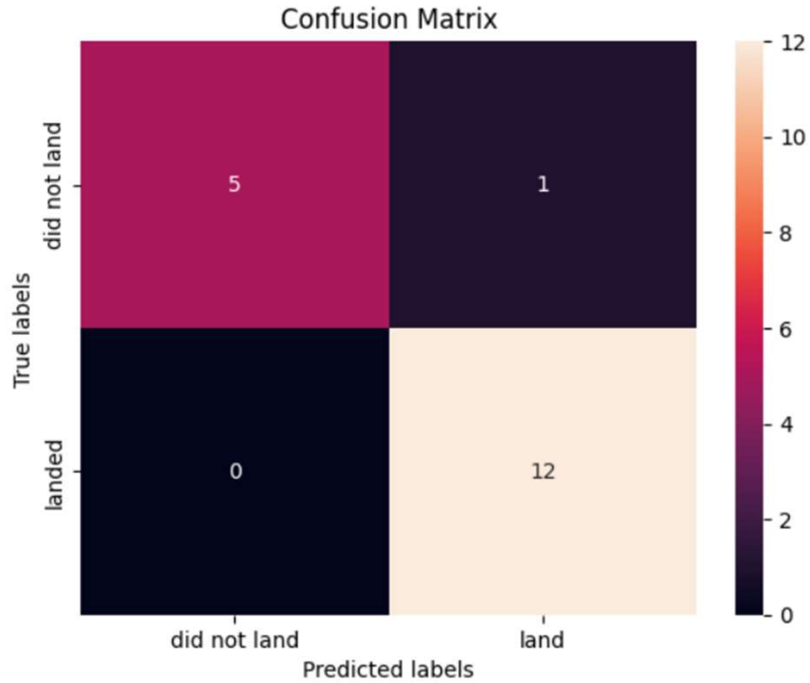
Predictive Analysis (Classification)

Classification Accuracy



The bar chart illustrates the accuracy of each machine learning model, revealing that all four models demonstrate similar performance. Nevertheless, the Tree Model stands out with the highest accuracy among them.

Confusion Matrix



The confusion matrix analysis reveals that the Tree model, employing the best algorithm, achieves the strongest performance with a score of 0.88

Conclusions

The conclusions drawn from the analysis are as follows:

- **Location Strategy**: Launch sites are commonly positioned near coastlines, allowing rockets to be launched over the ocean, thus minimizing potential hazards to densely populated areas.
- **Success Rate of Sites**: KSC LC-39A ranks as the most successful site, with an impressive success rate of 76.9%.
- **Payload Success Achievement**: The payload mass range between 2,000 and 6,000 KG consistently achieves high success rates, as evidenced by numerous data plots in this category.
- **Performance of Machine Learning Models**: Although the models generally display similar accuracy, the Tree Model is notably superior, achieving a top performance score of 0.88 according to analysis.

The findings emphasize the vital role of choosing launch sites strategically and adopting advanced modelling methods to enhance both launch success rates and predictive accuracy.


Thank you!